

Массивы и указатели.

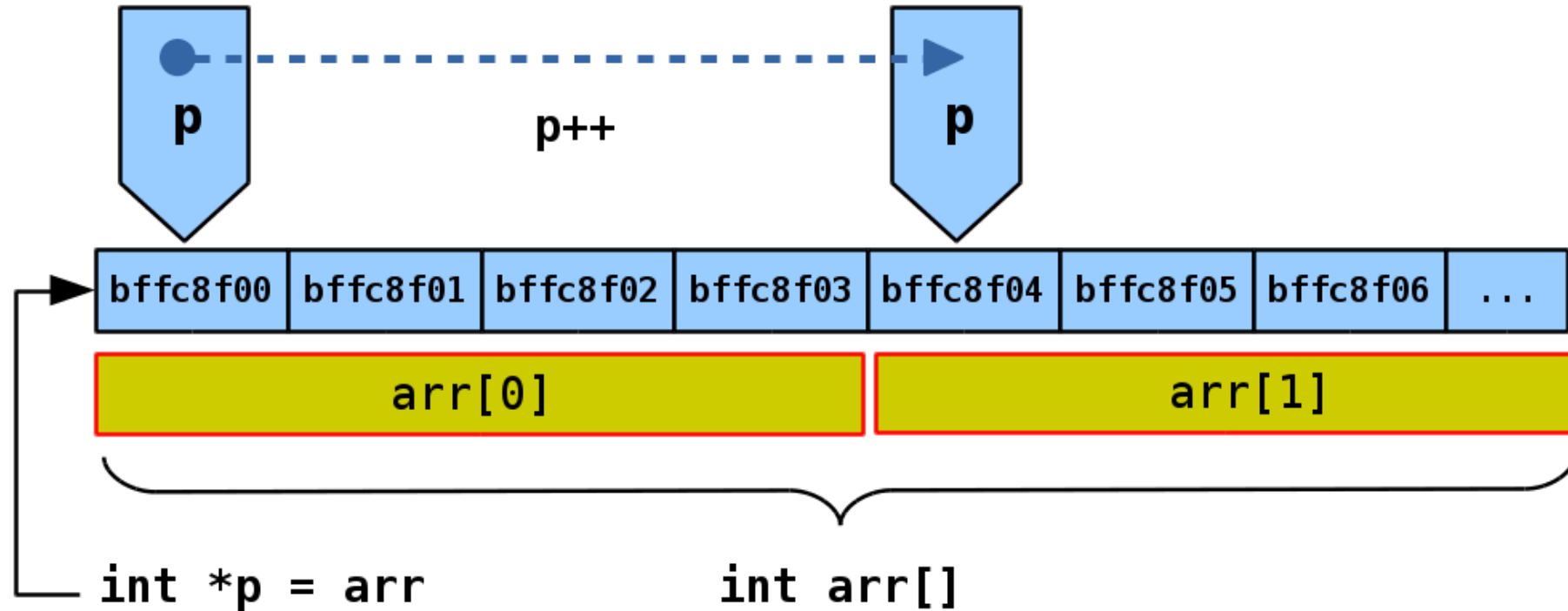
Динамически создаваемые массивы

Массивы и указатели

- Массивы и указатели тесно связаны.
- **Имя массива является указателем-константой**, значением которой служит адрес первого элемента массива (`&arr[0]`).
- Следовательно, имя массива может являться инициализатором указателя к которому будут применимы все правила адресной арифметики, связанной с указателями.

- Выражение `arr[i]` — обращение к элементу по индексу соответствует выражению $*(arr + i)$, которое называется **указателем-смещением**.
- Это выражение более наглядно иллюстрирует, как C++ на самом деле работает с элементами массива.
- Переменная-счетчик `i` указывает на сколько элементов необходимо сместиться от первого элемента.

Иллюстрация смещения



Преимущества использования указателей при работе с элементами массива

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      const int n = 2000000;
6      int mass[n] {};
7
8      for (int i = 0; i < n; i++) {
9          mass[i] = i;
10         cout << mass[i];
11     }
12     return 0;
13 }
```

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      const int n = 2000000;
6      int mass[n] {};
7
8      int *p = mass;
9      for (int i = 0; i < n; i++) {
10         *p = i;
11         cout << *p++;
12     }
13     return 0;
14 }
```

Выход за границы массива

- В языке C++ отсутствует **контроль соблюдения выхода за границы C-массива**.
- Ответственность за соблюдение режима обработки элементов в пределах границ массива лежит целиком на разработчике алгоритма.

Динамическое выделение памяти в Си

- Для использования функций динамического выделения памяти необходимо описать **указатель**, представляющий собой начальный адрес хранения элементов массива.

```
int *p; // указатель на тип int
```

- Начальный адрес статического массива определяется компилятором **в момент его объявления и не может быть изменен.**
- Для динамического массива **начальный адрес присваивается объявленному указателю на массив** в процессе выполнения программы.

Стандартные функции динамического выделения памяти

```
void* malloc(РазмерМассиваВБайтах);
```

```
void* calloc(ЧислоЭлементов, РазмерЭлементаВБайтах);
```

- Для использования функций динамического распределения памяти необходимо подключение библиотеки:

```
#include <malloc.h>
```


- Память, динамически выделенная с использованием функций `calloc()`, `malloc()`, может быть освобождена с использованием функции

`free(указатель);`

- «Правилом хорошего тона» в программировании является освобождение динамически выделенной памяти в случае отсутствия ее дальнейшего использования.
- Однако если динамически выделенная память не освобождается явным образом, она будет освобождена по завершении выполнения программы.

```
printf("Введите размер массива: ");
scanf("%d", &n);
// Выделение памяти
a = (int*)malloc(n * sizeof(int));
// Ввод элементов массива
for (i = 0; i<n; i++)
{
    printf("a[%d] = ", i);
    scanf("%d", &a[i]);
}
// Вывод элементов массива
for (i = 0; i<n; i++)
    printf("%d ", a[i]);
free(a);
```

Операции new и delete

- С++ использует новые методы работы с динамической памятью при помощи операторов new и delete.
- Синтаксис:

`<тип_данных> *имя_указателя = new тип_данных;`

- Например:

```
int *a = new int; // Объявление указателя типа int
```

```
int *b = new int(5); // Инициализация указателя
```

Как работает new

- Правая часть выражения говорит о том, что **new** запрашивает блок памяти для хранения данных определенного типа.
- Если память будет найдена, то возвращается адрес, который присваивается переменной-указателем, имеющей заданный тип.
- **Теперь получить доступ к динамически созданной памяти можно только с помощью указателей!**

delete

- После выполнения работы с выделенной памятью ее необходимо освободить (вернуть, сделать доступной для других данных) с помощью операции **delete**.
- Ошибки, при которых память не освобождается, приводят к "утечкам памяти", что, в свою очередь, может привести к аварийному завершению программы.
- Операция delete может применяться к нулевому указателю (nullptr) или созданному с помощью new (**new и delete используются в паре!**).

Динамические массивы

- Динамический массив - это массив, размер которого определяется в процессе работы программы.
- Строго говоря C-массив не является динамическим в C++. То есть, можно определять только размер массива, а изменение размера массива, в процессе работы программы, по-прежнему невозможно.
- Для получения массива нужного размера необходимо выделять память под новый массив и копировать в него данные из исходного, а затем освободить память выделенную ранее под исходный массив

Выделение и освобождение памяти под массив

- Синтаксис выделения памяти для массива имеет вид
указатель = new тип[размер].
- В качестве размера массива может выступать любое целое положительное значение.

```
int* arr = new int[n];
```

...

```
delete[] arr;
```

- При этом размер массива не указывается.